17 General purpose direct memory access controller (GPDMA)

17.1 GPDMA introduction

The general purpose direct memory access (GPDMA) controller is a bus master and system peripheral.

The GPDMA is used to perform programmable data transfers between memory-mapped peripherals and/or memories via linked-lists, upon the control of an off-loaded CPU.

17.2 GPDMA main features

- Dual bidirectional AHB master
- Memory-mapped data transfers from a source to a destination:
  - Peripheral-to-memory
  - Memory-to-peripheral
  - Memory-to-memory
  - Peripheral-to-peripheral
- Autonomous data transfers during low-power modes (see *Section 17.3.2*)

  Transfer arbitration based on a 4-grade programmed priority at channel level:
  - One high-priority traffic class, for time-sensitive channels (queue 3)
  - Three low-priority traffic classes, with a weighted round-robin allocation for non time-sensitive channels (queues 0, 1, 2)
- Per channel event generation, on any of the following events: transfer complete, half transfer complete, data transfer error, user setting error, link transfer error, completed suspension, and trigger overrun
- Per channel interrupt generation, with separately programmed interrupt enable per event
- 16 concurrent GPDMA channels:
  - Per channel FIFO for queuing source and destination transfers (see *Section 17.3.1*)
  - Intra-channel GPDMA transfers chaining via programmable linked-list into memory, supporting two execution modes: run-to-completion and link step mode
  - Intra-channel and inter-channel GPDMA transfers chaining via programmable GPDMA input triggers connection to GPDMA task completion events
- Per linked-list item within a channel:
  - Separately programmed source and destination transfers
  - Programmable data handling between source and destination: byte-based reordering, packing or unpacking, padding or truncation, sign extension and left/right realignment
  - Programmable number of data bytes to be transferred from the source, defining the block level

  - Linear source and destination addressing: either fixed or contiguously incremented addressing, programmed at a block level, between successive burst transfers
  - 2D source and destination addressing: programmable signed address offsets between successive burst transfers (non-contiguous addressing within a block, combined with programmable signed address offsets between successive blocks, at a second 2D/repeated block level, for a reduced set of channels (see *Section 17.3.1*)
  - Support for scatter-gather (multi-buffer transfers), data interleaving and deinterleaving via 2D addressing
  - Programmable GPDMA request and trigger selection
  - Programmable GPDMA half transfer and transfer complete events generation
  - Pointer to the next linked-list item and its data structure in memory, with automatic update of the GPDMA linked-list control registers
- Debug:
  - Channel suspend and resume support
  - Channel status reporting, including FIFO level, and event flags
- TrustZone support:
  - Support for secure and nonsecure GPDMA transfers, independently at a first channel level, and independently at a source/destination and link sublevels
  - Secure and nonsecure interrupts reporting, resulting from any of the respectively secure and nonsecure channels
  - TrustZone-aware AHB slave port, protecting any GPDMA secure resource (register, register field) from a nonsecure access
- Privileged/unprivileged support:
  - Support for privileged and unprivileged GPDMA transfers, independently at a channel level
  - Privileged-aware AHB slave port

17.3 GPDMA implementation

17.3.1 GPDMA channels

A given GPDMA channel x is implemented with the following features and intended usage. To make the best use of the GPDMA performances, the table below lists some general recommendations, allowing the user to select and allocate a channel, given its implemented FIFO size and the requested GPDMA transfer.

Table 134. GPDMA1 channel implementation

Channel x	Hardware parameters		Features
	dma_fifo_size[x]	dma_addressing[x]	
x = 0 to 11	2	0	Channel x (x = 0 to 11) is implemented with: – a FIFO of 8 bytes, 2 words – fixed/contiguously incremented addressing These channels must be typically allocated for GPDMA transfers between an APB or AHB peripheral and SRAM.
x = 12 to 15	4	1	Channel x (x = 12 to 15) is implemented with: – a FIFO of 32 bytes, 8 words – 2D addressing These channels may be also used for GPDMA transfers, between a demanding AHB peripheral and SRAM, or for transfers from/to external memories.

17.3.2 GPDMA autonomous mode in low-power modes

The GPDMA autonomous mode and wake-up feature are implemented in the device low-power modes as per the table below.

Table 135. GPDMA1 autonomous mode and wake-up in low-power modes

Feature	Low-power modes
Autonomous mode and wake-up	GPDMA1 in Sleep, Stop 0 and Stop 1 modes

17.3.3 GPDMA requests

A GPDMA request from a peripheral can be assigned to a GPDMA channel x, via REQSEL[6:0] in GPDMA_CxTR2, provided that SWREQ = 0.

The GPDMA requests mapping is specified in the table below.

Table 136. Programmed GPDMA1 request

GPDMA_CxTR2.REQSEL[6:0]	Selected GPDMA request
0	adc1_dma
1	adc4_dma
2	dac1_ch1_dma
3	dac1_ch2_dma
4	tim6_upd_dma
5	tim7_upd_dma
6	spi1_rx_dma
7	spi1_tx_dma

Table 136. Programmed GPDMA1 request (continued)

GPDMA_CxTR2.REQSEL[6:0]	Selected GPDMA request
8	spi2_rx_dma
9	spi2_tx_dma
10	spi3_rx_dma
11	spi3_tx_dma
12	i2c1_rx_dma
13	i2c1_tx_dma
14	i2c1_evc_dma
15	i2c2_rx_dma
16	i2c2_tx_dma
17	i2c2_evc_dma
18	i2c3_rx_dma
19	i2c3_tx_dma
20	i2c3_evc_dma
21	i2c4_rx_dma
22	i2c4_tx_dma
23	i2c4_evc_dma
24	usart1_rx_dma
25	usart1_tx_dma
26	usart2_rx_dma
27	usart2_tx_dma
28	usart3_rx_dma
29	usart3_tx_dma
30	uart4_rx_dma
31	uart4_tx_dma
32	uart5_rx_dma
33	uart5_tx_dma
34	lpuart1_rx_dma
35	lpuart1_tx_dma
36	sai1_a_dma
37	sai1_b_dma
38	sai2_a_dma
39	sai2_b_dma
40	octospi1_dma
41	octospi2_dma
42	tim1_cc1_dma

Table 136. Programmed GPDMA1 request (continued)

GPDMA_CxTR2.REQSEL[6:0]	Selected GPDMA request
43	tim1_cc2_dma
44	tim1_cc3_dma
45	tim1_cc4_dma
46	tim1_upd_dma
47	tim1_trg_dma
48	tim1_com_dma
49	tim8_cc1_dma
50	tim8_cc2_dma
51	tim8_cc3_dma
52	tim8_cc4_dma
53	tim8_upd_dma
54	tim8_trg_dma
55	tim8_com_dma
56	tim2_cc1_dma
57	tim2_cc2_dma
58	tim2_cc3_dma
59	tim2_cc4_dma
60	tim2_upd_dma
61	tim3_cc1_dma
62	tim3_cc2_dma
63	tim3_cc3_dma
64	tim3_cc4_dma
65	tim3_upd_dma
66	tim3_trg_dma
67	tim4_cc1_dma
68	tim4_cc2_dma
69	tim4_cc3_dma
70	tim4_cc4_dma
71	tim4_upd_dma
72	tim5_cc1_dma
73	tim5_cc2_dma
74	tim5_cc3_dma
75	tim5_cc4_dma
76	tim5_upd_dma
77	tim5_trg_dma

Table 136. Programmed GPDMA1 request (continued)

GPDMA_CxTR2.REQSEL[6:0]	Selected GPDMA request
78	tim15_cc1_dma
79	tim15_upd_dma
80	tim15_trg_dma
81	tim15_com_dma
82	tim16_cc1_dma
83	tim16_upd_dma
84	tim17_cc1_dma
85	tim17_upd_dma
86	dcmi_dma or pssi_dma(1)
87	aes_in_dma
88	aes_out_dma
89	hash_in_dma
90	ucpd1_tx_dma
91	ucpd1_rx_dma
92	mdf1_flt0_dma
93	mdf1_flt1_dma
94	mdf1_flt2_dma
95	mdf1_flt3_dma
96	mdf1_flt4_dma
97	mdf1_flt5_dma
98	adf1_flt0_dma
99	fmac_read_dma
100	fmac_write_dma
101	cordic_read_dma
102	cordic_write_dma
103	saes_in_dma
104	saes_out_dma
105	lptim1_ic1_dma
106	lptim1_ic2_dma
107	lptim1_ue_dma
108	lptim2_ic1_dma
109	lptim2_ic2_dma
110	lptim2_ue_dma
111	lptim3_ic1_dma
112	lptim3_ic2_dma

Table 136. Programmed GPDMA1 request (continued)

GPDMA_CxTR2.REQSEL[6:0]	Selected GPDMA request
113	lptim3_ue_dma
114	hspi1_dma
115	i2c5_rx_dma
116	i2c5_tx_dma
117	i2c5_evc_dma
118	i2c6_rx_dma
119	i2c6_tx_dma
120	i2c6_evc_dma
121	usart6_rx_dma
122	usart6_tx_dma
123	adc2_dma
124	jpeg_rx_dma
125	jpeg_tx_dma

1. Depends on which exclusive function is used.

17.3.4 GPDMA block requests

Some GPDMA requests must be programmed as a block request, and not as a burst request. Then BREQ in GPDMA_CxTR2 must be set for a correct GPDMA execution of the requested peripheral transfer at the hardware level.

Table 137. Programmed GPDMA1 request as a block request

GPDMA block requests
lptim1_ue_dma
lptim3_ue_dma

17.3.5 GPDMA triggers

A GPDMA trigger can be assigned to a GPDMA channel x, via TRIGSEL[6:0] in GPDMA_CxTR2, provided that TRIGPOL[1:0] defines a rising or a falling edge of the selected trigger (TRIGPOL[1:0] = 01 or TRIGPOL[1:0] = 10).

Table 138.Programmed GPDMA1 trigger

GPDMA_CxTR2.TRIGSEL[6:0]	Selected GPDMA trigger
0	exti0
1	exti1
2	exti2
3	exti3

Table 138.Programmed GPDMA1 trigger (continued)

GPDMA_CxTR2.TRIGSEL[6:0]	Selected GPDMA trigger
4	exti4
5	exti5
6	exti6
7	exti7
8	tamp_trg1
9	tamp_trg2
10	tamp_trg3
11	lptim1_ch1
12	lptim1_ch2
13	lptim2_ch1
14	lptim2_ch2
15	lptim4_out
16	comp1_out
17	comp2_out
18	rtc_alra_trg
19	rtc_alrb_trg
20	rtc_wut_trg
21	reserved
22	gpdma1_ch0_tc
23	gpdma1_ch1_tc
24	gpdma1_ch2_tc
25	gpdma1_ch3_tc
26	gpdma1_ch4_tc
27	gpdma1_ch5_tc
28	gpdma1_ch6_tc
29	gpdma1_ch7_tc
30	gpdma1_ch8_tc
31	gpdma1_ch9_tc
32	gpdma1_ch10_tc
33	gpdma1_ch11_tc
34	gpdma1_ch12_tc
35	gpdma1_ch13_tc
36	gpdma1_ch14_tc
37	gpdma1_ch15_tc
38	lpdma1_ch0_tc

Table 138.Programmed GPDMA1 trigger (continued)

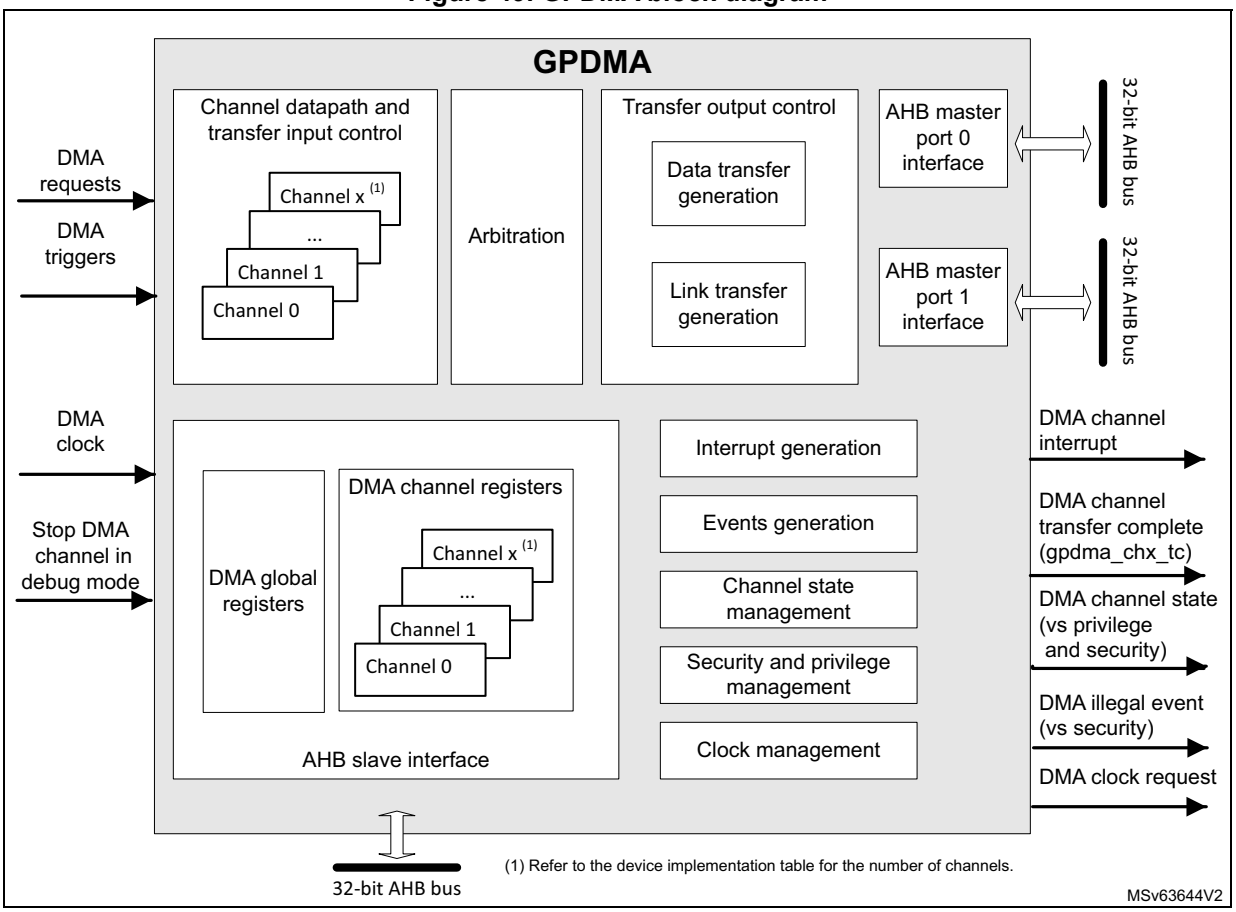
GPDMA_CxTR2.TRIGSEL[6:0]	Selected GPDMA trigger
39	lpdma1_ch1_tc
40	lpdma1_ch2_tc
41	lpdma1_ch3_tc
42	tim2_trgo
43	tim15_trgo
44	tim3_trgo(1)
45	tim4_trgo(1)
46	tim5_trgo(1)
47	ltdc_li
48	dsi_te
49	dsi_er
50	dma2d_tc
51	dma2d_ctc
52	dma2d_tw
53	gpu2d_flag[0]
54	gpu2d_flag[1]
55	gpu2d_flag[2]
56	gpu2d_flag[3]
57	adc4_awd1
58	adc1_awd1
59	gfxtim_ev4
60	gfxtim_ev3
61	gfxtim_ev2
62	gfxtim_ev1
63	jpeg_eoc_trg
64	jpeg_ifnf_trg
65	jpeg_ift_trg
66	jpeg_ofne_trg
67	jpeg_oft_trg

1. Connections only present in STM32U59x/5Ax and STM32U5Fx/5Gx.

17.4 GPDMA functional description

17.4.1 GPDMA block diagram

Figure 49. GPDMA block diagram



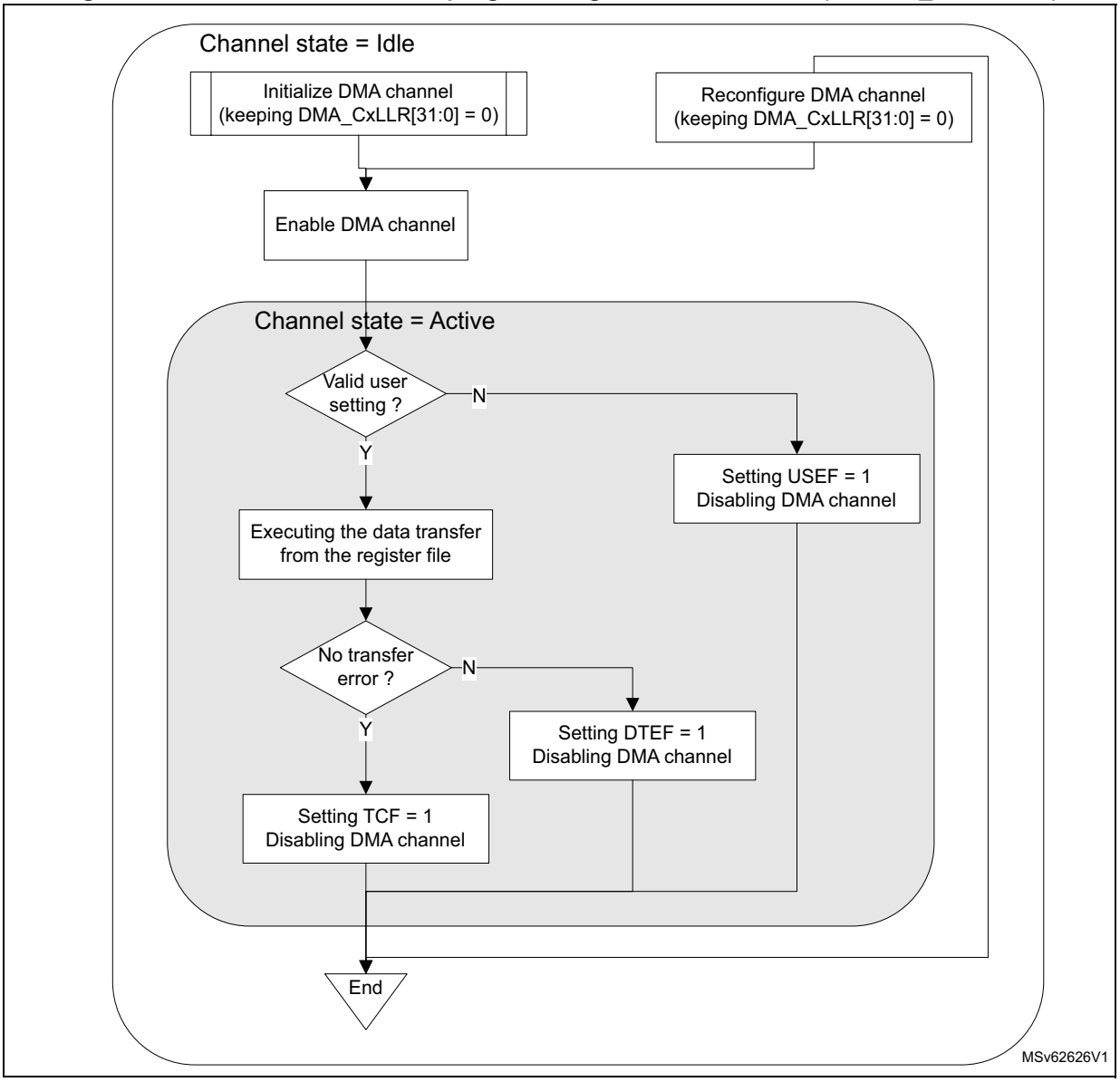
17.4.2 GPDMA channel state and direct programming without any linked-list

After a GPDMA reset, a GPDMA channel x is in idle state. When the software writes 1 in GPDMA_CxCR.EN, the channel takes into account the value of the different channel configuration registers (GPDMA_CxXXX), switches to the active/non-idle state and starts to execute the corresponding requested data transfers.

After enabling/starting a GPDMA channel transfer by writing 1 in GPDMA_CxCR.EN, a GPDMA channel interrupt on a complete transfer notifies the software that the GPDMA channel is back in idle state (EN is then deasserted by hardware) and that the channel is ready to be reconfigured then enabled again.

The figure below illustrates this GPDMA direct programming without any linked-list (GPDMA_CxLLR = 0).

Figure 50. GPDMA channel direct programming without linked-list (GPDMA_CxLLR = 0)



17.4.3 GPDMA channel suspend and resume

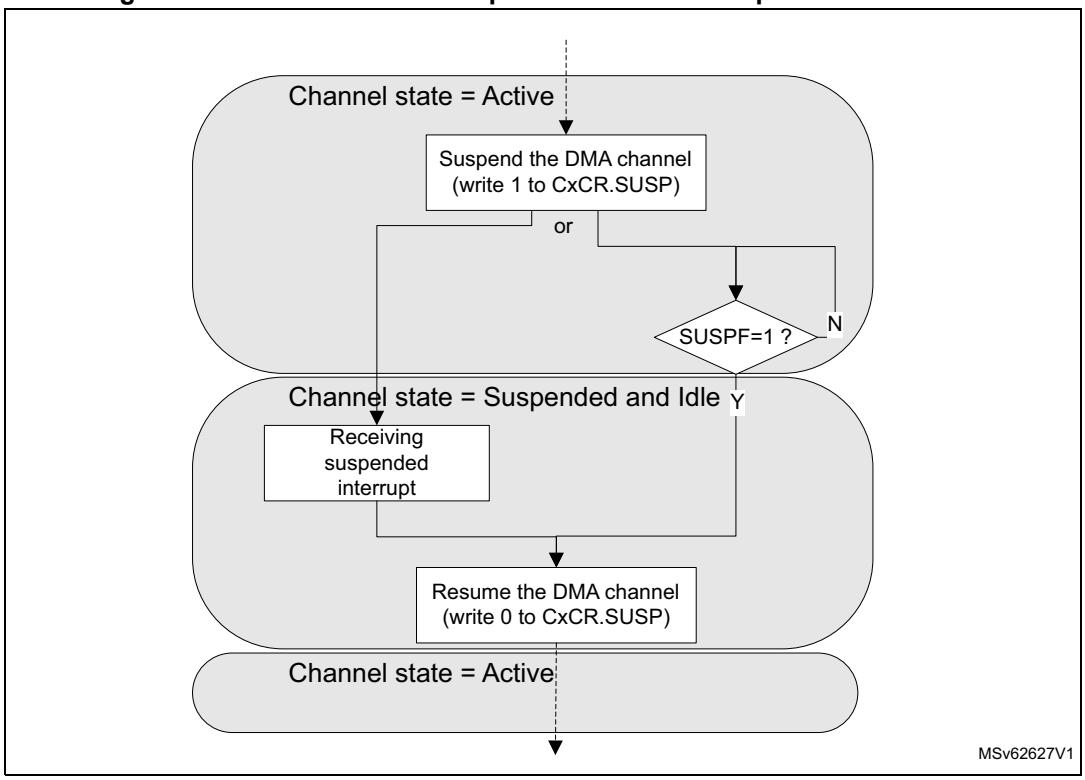
The software can suspend on its own a channel still active, with the following sequence:

1. The software writes 1 into the GPDMA_CxCR.SUSP bit.
2. The software polls the suspended flag GPDMA_CxSR.SUSPF until SUSPF = 1, or waits for an interrupt previously enabled by writing 1 to GPDMA_CxCR.SUSPIE. Wait for the channel to be effectively in suspended state means wait for the completion of any ongoing GPDMA transfer over its master ports. Then the software can observe, in a steady state, any read register or register field that is hardware modifiable.

Note that an ongoing GPDMA transfer can be a data transfer (a source/destination burst transfer) or a link transfer for the internal update of the linked-list register file from the next linked-list item.

3. The software safely resumes the suspended channel by writing 0 to GPDMA_CxCR.SUSP.

Figure 51. GPDMA channel suspend and resume sequence



Note: A suspend and resume sequence does not impact the GPDMA_CxCR.EN bit. Suspending a channel (transfer) does not suspend a started trigger detection.

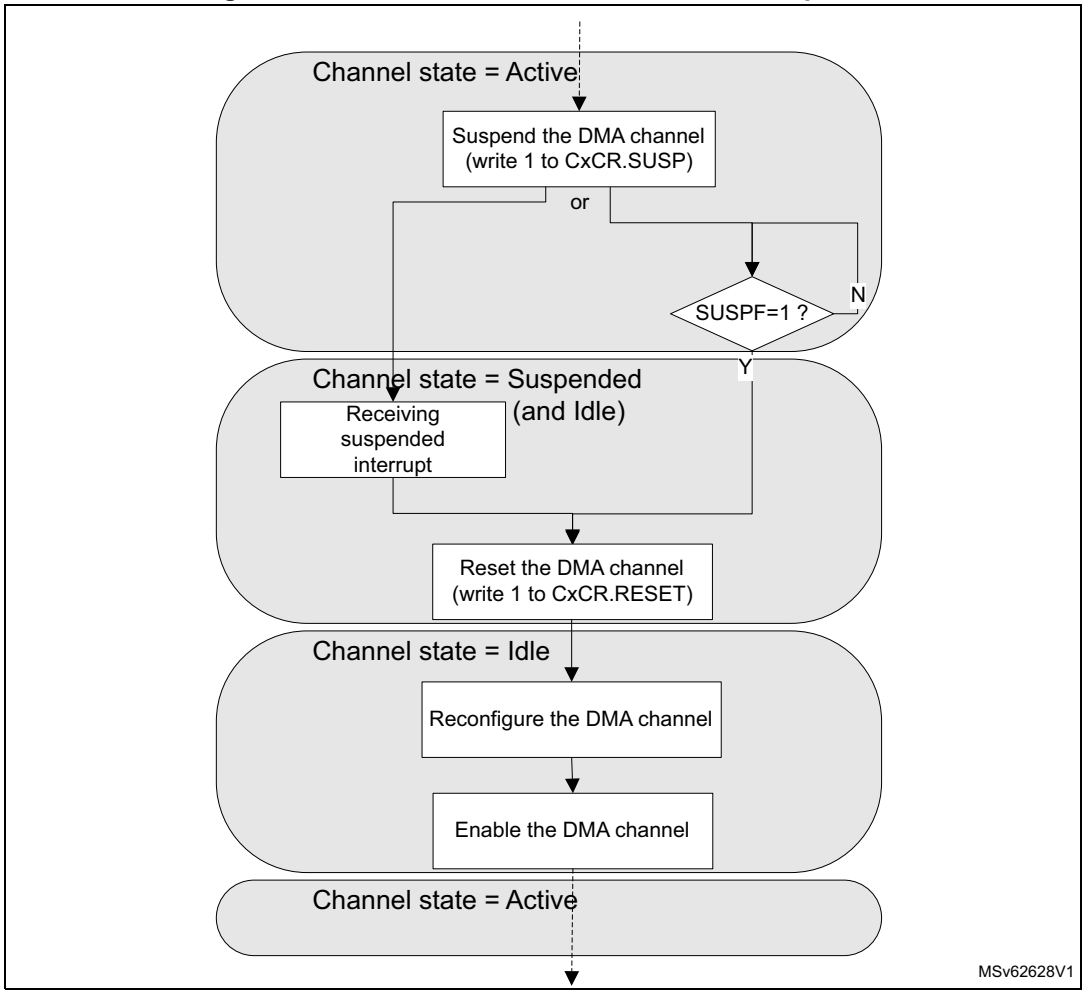
17.4.4 GPDMA channel abort and restart

Alternatively, like for aborting a continuous GPDMA transfer with a circular buffering or a double buffering, the software can abort, on its own, a still active channel with the following sequence:

1. The software writes 1 into the GPDMA_CxCR.SUSP bit.
2. The software polls suspended flag GPDMA_CxSR.SUSPF until SUSPF = 1, or waits for an interrupt previously enabled by writing 1 to GPDMA_CxCR.SUSPIE. Wait for the channel to be effectively in suspended state means wait for the completion of any ongoing GPDMA transfer over its master port.
3. The software resets the channel by writing 1 to GPDMA_CxCR.RESET. This causes the reset of the FIFO, the reset of the channel internal state, the reset of the GPDMA_CxCR.EN bit, and the reset of the GPDMA_CxCR.SUSP bit.
4. The software safely reconfigures the channel. The software must reprogram the hardware-modified GPDMA_CxBR1, GPDMA_CxSAR, and GPDMA_CxDAR registers.

5. In order to restart the aborted then reprogrammed channel, the software enables it again by writing 1 to the GPDMA_CxCR.EN bit.

Figure 52. GPDMA channel abort and restart sequence



17.4.5 GPDMA linked-list data structure

Alternatively to the direct programming mode, a channel can be programmed by a list of transfers, known as a list of linked-list items (LLI). Each LLI is defined by its data structure.

The base address in memory of the data structure of a next LLI_{n+1} of a channel x is the sum of the following:

- the link base address of the channel x (in GPDMA_CxLBAR)
- the link address offset (LA[15:2] field in GPDMA_CxLLR). The linked-list register GPDMA_CxLLR is the updated result from the data structure of the previous LLIn of the channel x.

The data structure for each LLI may be specific.

A linked-list data structure is addressed following the value of the UT1, UT2, UB1, USA, UDA and ULL bits, plus UB2 and UT3, in GPDMA_CxLLR.

In linked-list mode, each GPDMA linked-list register (GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR or GPDMA_CxLLR, plus GPDMA_CxTR3 or GPDMA_CxBR2) is conditionally and automatically updated from the next linked-list data structure in the memory, following the current value of the GPDMA_CxLLR register that was conditionally updated from the linked-list data structure of the previous LLI.

Static linked-list data structure

For example, when the update bits (UT1, UT2, UB1, USA, UDA and ULL, plus UB2 and UT3) in GPDMA_CxLLR are all asserted, the linked-list data structure in the memory is maximal with:

- channel x (x = 0 to 11) contiguous 32-bit locations, including GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR and GPDMA_CxLLR (see *Figure 53*) and including the first linked-list register file (LLI_0) and the next LLIs (such as LLI_1, LLI_2) in the memory
- channel x(x = 12 to 15), contiguous 32-bit locations, including GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR, and GPDMA_CxLLR, plus GPDMA_CxTR3 and GPDMA_CxBR2 (see *Figure 54*), and including the first linked-list register file (LLI_0) and the next LLIs (such as LLI_1, LLI_2) in the memory

Figure 53. Static linked-list data structure (all Uxx = 1) of a linear addressing channel x

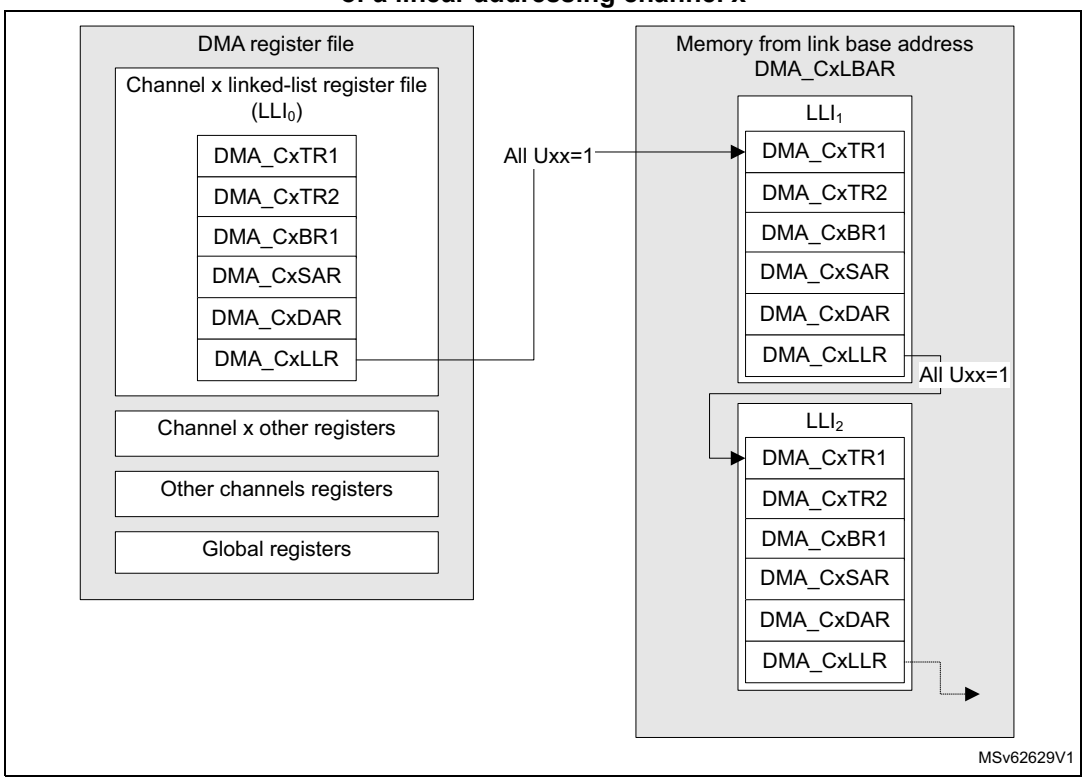
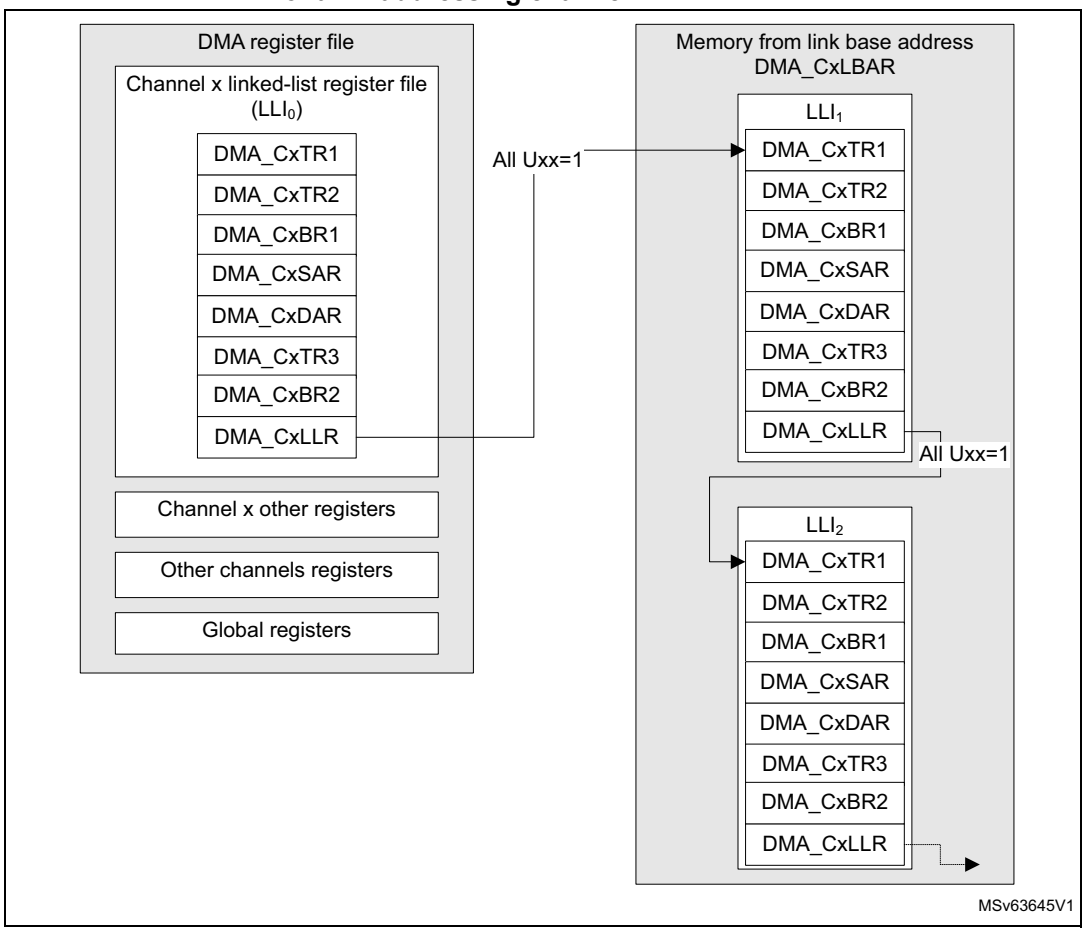


Figure 54. Static linked-list data structure (all Uxx = 1) of a 2D addressing channel x



Dynamic linked-list data structure

Alternatively, the memory organization for the full list of LLIs can be compacted with specific data structure for each LLI.

If UT1 = 0 and UT2 = 1, the link address offset of the register GPDMA_CxLLR is pointing to the updated value of the GPDMA_CxTR2 instead of the GPDMA_CxTR1 which is not to be modified (see *Figure 55*).

Example: if UT1 = UB1 = USA = 0 and if UT3 = UB2 = 0, when channel x is with 2D addressing, and if UT2 = UDA = ULL = 1, the next LLI does not contain an (updated) value for GPDMA_CxTR1, nor GPDMA_CxBR1, nor GPDMA_CxSAR, nor GPDMA_CxTR3, nor GPDMA_CxBR2 when channel x is with 2D addressing. The next LLI contains an updated value for GPDMA_CxTR2, GPDMA_CxDAR, and GPDMA_CxLLR, as shown in *Figure 56*.